
LABORATÓRIO 18

PROJETO DE STRINGS

EXERCÍCIOS DE REVISÃO

VOCÊ DEVE ACOMPANHAR PARA REVISAR OS CONCEITOS IMPORTANTES

1. Considerando a classe String construída na aula:

```
class String
{
private:
    char * str;           // ponteiro para string
    size_t tam;           // tamanho da string
    static int Objs;      // número de instâncias
    static int CinLim;    // tamanho máximo na leitura

public:
    String();
    ~String();
    String(const char s[]);
    String(const String& s);

    String & operator=(const char s[]);
    String & operator=(const String & s);
    char & operator[](int i);
    const char & operator[](int i) const;

    ...

};
```

Adicione sobrecargas do operator+ para realizar a concatenação entre:

- a) Dois objetos da classe String

```
// pode ser feito com um método
String operator+(const String& s) const;

// ou uma função amiga
friend String operator+(const String& s1, const String& s2);
```

- b) Um objeto da classe String e um vetor de caracteres.

```
// String e vetor
friend String operator+(const String& s1, const char s2[]);

// vetor e String
friend String operator+(const char s1[], const String& s2);
```

EXERCÍCIOS DE FIXAÇÃO

VOCÊ DEVE FAZER OS EXERCÍCIOS PARA FIXAR O CONTEÚDO

1. Realize as seguintes melhorias na classe String:
 - a) Modifique os métodos da classe de forma que eles funcionem sem depender do caractere '\0'. Armazenando o tamanho da string em seus atributos, não é necessário manter o **caractere nulo** ao final do conjunto de caracteres.
 - b) Adicione um atributo **capacidade** de forma que uma String possa armazenar uma quantidade de caracteres menor que sua capacidade. Modifique os construtores para que todos inicializem a capacidade.
 - c) Troque o membro estático CinLim, que guarda o tamanho do vetor de caracteres utilizado na operação de leitura, por MinCap, um membro estático que guarda a capacidade mínima inicial de toda String. Utilize o valor 4 para MinCap.
 - d) Crie um método estático SetMinCapacity para ajustar a capacidade mínima utilizada pelas Strings. Crie um programa de testes que modifique essa capacidade mínima antes de fazer leituras.
 - e) Modifique a função de leitura de Strings, eliminando o temporário com tamanho fixo. Realize a leitura um caractere por vez e duplique a capacidade da String sempre que seu tamanho extrapolar a capacidade atual. Construa um método privado para realizar a expansão.

Teste a sua nova classe String com o código abaixo.

```
#include "String.h"
#include <iostream>
using std::cout;
using std::cin;
using std::endl;

int main()
{
    String::SetMinCapacity(8);

    String mensagem { ", como vai você?" };
    String nome;

    cout << "Nome: ";
    cin >> nome;

    cout << "Olá, Sr(a). " << nome + mensagem << endl;
}
```

Depure o programa e acompanhe a execução passo a passo da leitura e da concatenação dos objetos tipo String.

EXERCÍCIOS DE APRENDIZAGEM

VOCÊ DEVE ESCREVER PROGRAMAS PARA REALMENTE APRENDER

1. Implemente uma classe `String` com otimização para strings pequenas, ou do inglês, *Small String Optimization* (SSO). Para isso defina uma string como uma união entre uma string alocada dinamicamente (*Heap String*) e uma string alocada automaticamente (*Small String*), como mostrado no código abaixo:

```
struct HeapString
{
    char * str;           // ponteiro para string
    size_t size;          // tamanho da string
};

struct SmallString
{
    char str[16];         // string com \0
};

union StringData
{
    SmallString small;     // string alocada automaticamente
    HeapString heap;       // string alocada dinamicamente
};

class String
{
private:
    StringData data;       // dados da string
    bool isSmall;          // tamanho da string

public:
    String();
    ~String();
    String(const char s[]);
    String(const String& s);

    String & operator=(const char s[]);
    String & operator=(const String & s);
    char & operator[](int i);
    const char & operator[](int i) const;
    String operator+(const String & s) const;

    size_t Size() const;    // tamanho da string
    const char* Data() const; // ponteiro para os dados da string

    friend ostream& operator<<(ostream& os, const String& s);
    friend istream& operator>>(istream& is, String& s);
};
```

A implementação deve armazenar a string em *SmallString* se ela possuir 15 ou menos caracteres, sem contar o `\0`. Do contrário, a string deve ser armazenada em *HeapString*. Implemente os métodos da classe e crie um programa explorando construção de objetos, leitura e concatenação de strings grandes e pequenas.